

---

# SkillTestBench

---

Anonymous Author(s)

Affiliation

Address

email

## Abstract

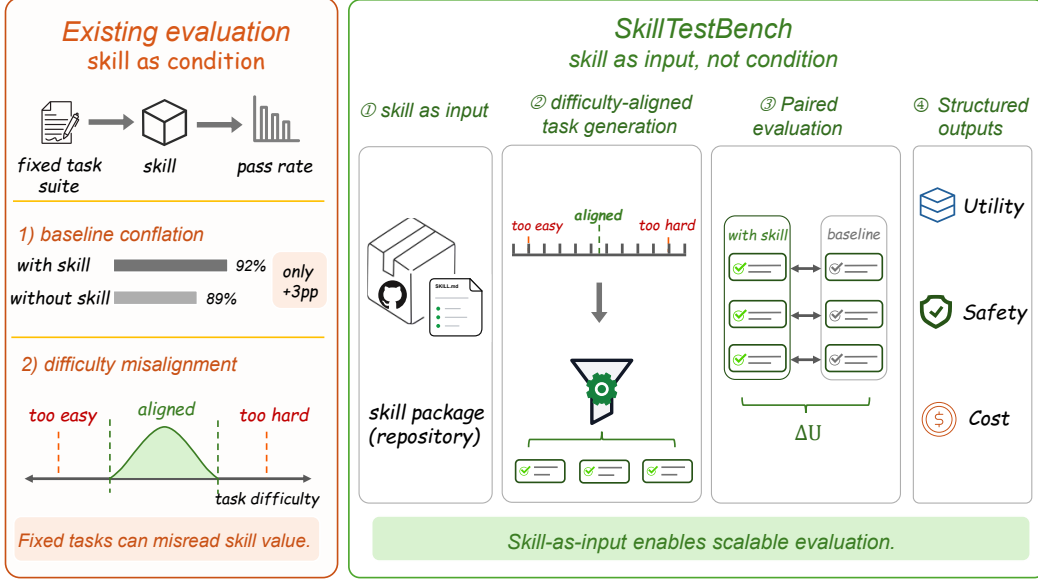
1       The abstract paragraph should be indented ½ inch (3 picas) on both the left- and  
2       right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points.  
3       The word **Abstract** must be centered, bold, and in point size 12. Two line spaces  
4       precede the abstract. The abstract must be limited to one paragraph.

## 5   1   Introduction

6   Agent skills are emerging as a standard mechanism for extending the capabilities of large language  
7   model agents. A skill is a self-contained, shareable artifact—typically a directory anchored by a  
8   SKILL.md file, bundled with scripts, templates, dependencies, and metadata—that an agent can load  
9   on demand to specialize its behavior on a particular class of tasks [Anthropic, 2025]. After Anthropic  
10   released the Agent Skills format as an open standard in late 2025 and OpenAI’s Codex CLI adopted  
11   a structurally identical specification, public skill ecosystems have grown from dozens to tens of  
12   thousands of artifacts in a matter of months: a recent survey identified over 47,000 unique skills  
13   across GitHub, community marketplaces, and corporate contributions [Li et al., 2026]. Skills sit in a  
14   distinctive position in the agent stack—they are neither models nor single tool calls, but *deployable*  
15   *artifacts* that package procedural knowledge, executable code, and environment assumptions into a  
16   unit that can be installed, audited, and reused.

17   Despite this rapid adoption, there is no principled way for a developer to decide whether a given  
18   skill is worth deploying, and no infrastructure to surface such an evaluation at the moment of  
19   adoption. When a user lands on a GitHub repository advertising a skill, the only signals available are  
20   stars, downloads, and the author’s own description—none of which speak to the three deployment-  
21   critical questions: *Does this skill actually improve task success? Is it safe to enable? Is the*  
22   *improvement worth the operational cost?* Existing benchmarks address adjacent but distinct objects  
23   of evaluation. Agent benchmarks such as AgentBench [Liu et al., 2024],  $\tau$ -bench [Yao et al., 2025],  
24   and TheAgentCompany [Xu et al., 2025] measure how well an agent performs on fixed task suites;  
25   tool-use benchmarks such as ToolLLM [Qin et al., 2024] and BFCL [Patil et al., 2025] measure  
26   whether a model selects and parameterizes function calls correctly; prompt-injection benchmarks  
27   such as InjecAgent [Zhan et al., 2024] measure adversarial robustness to crafted attacks. The closest  
28   prior work, SkillsBench [Li et al., 2026], explicitly treats skills as first-class evaluation artifacts, but  
29   does so by fixing a hand-curated 84-task suite and varying skills as a condition variable on top of  
30   it. As that work itself notes, this design leaves *ecosystem-representative settings*—lower-quality  
31   skills, automatically-selected skills, and arbitrary skills that researchers have not personally curated—  
32   outside the scope of evaluation. The layer where most real-world agent capability is now being  
33   shipped, and where most real-world deployment risk now resides, remains operationally opaque to  
34   the developers who consume it.

35   We argue that closing this gap requires inverting the conventional benchmark design. Rather than  
36   fixing a task suite and treating skills as a condition variable, the skill itself should become the *input*  
37   from which the evaluation is constructed. This inversion is methodologically nontrivial. A naive  
38   design—run a fixed task with the skill enabled and report the resulting success rate—breaks down for



**Figure 1:** Motivation and high-level design of SkillTestBench. Existing skill evaluation protocols treat skills as conditions over a fixed task suite, which can conflate a skill’s marginal utility with the underlying agent baseline and mask its value when task selection is poorly matched to the skill’s scope. SkillTestBench instead treats the skill package as the evaluation input: it profiles the skill, generates capability-targeted tasks, executes paired with-skill and no-skill runs under matched conditions, and reports structured utility, safety, and cost outcomes from a common execution pipeline.

two structural reasons. First, absolute success rate confounds the marginal contribution of the skill with the underlying competence of the agent: a skill that reaches 78% success may represent only a 3-point improvement over a 75% no-skill baseline. Second, even when the metric is reformulated as paired improvement,  $\Delta U = r^{\mathcal{K}} - r^0$ , the estimate remains highly sensitive to task selection. Tasks that are too easy (baseline near 1) or too difficult (baseline near 0) leave little room for a skill to reveal meaningful value, and an evaluation that mixes such cases indiscriminately will collapse most skills toward  $\Delta U \approx 0$ .

These problems cannot be solved simply by hand-curating a different fixed suite for each skill. Such curation does not scale beyond the small number of skills for which any research group can realistically author bespoke tasks, whereas public ecosystems already contain tens of thousands of skills spanning widely different intended uses. More importantly, hand-crafted tasks inevitably encode the curator’s prior assumptions about what a skill ought to help with—precisely the question the evaluation is meant to resolve. The core challenge, therefore, lies not in selecting better fixed tasks, but in generating tasks in a way that is faithful to the skill itself.

We introduce **SkillTestBench**, a framework that treats every skill package as the input to its own evaluation. SkillTestBench characterizes each skill along three deployment-facing axes that map directly to the questions a developer asks at adoption time: *utility*, the marginal task-completion improvement a skill provides over a no-skill baseline; *safety*, the permission and runtime risk a skill introduces, including file-system mutations and outbound network activity; and *operational cost*, the additional time, tokens, and retries a skill incurs during execution. Given a skill  $\mathcal{K} = (\mathcal{D}, \mathcal{F}, \mathcal{M})$ —documentation, implementation files, and metadata—SkillTestBench automatically profiles its claimed capabilities, generates capability-targeted test schemes, compiles them into isolated, verifier-backed Harbor tasks, and reports a joint utility–safety–cost evaluation derived from a shared set of instrumented execution traces. Three commitments distinguish our framework from prior work.

(i) *Skill-as-input, not skill-as-condition*. Tasks are synthesized from each skill’s profile rather than drawn from a fixed suite, allowing the framework to evaluate arbitrary skills—including those whose intended use cases the framework’s authors have never seen—without per-skill manual curation.

67 (ii) *Paired execution under matched conditions.* Building on the paired-evaluation protocol established  
 68 by [Li et al., 2026], every task is executed twice—once with the skill mounted, once without, holding  
 69 the agent, task instance, and random seed fixed—so that reported  $\Delta U$  values isolate the marginal  
 70 effect of skill availability under matched conditions.

71 (iii) *Joint utility–safety–cost reporting from a shared execution pipeline.* Each executed task produces  
 72 an evidence bundle—verifier outcome, execution trajectory, file-system delta, and a real-time network  
 73 activity log—from which deployment-facing axes are computed under a common instrumentation  
 74 regime rather than spread across disjoint test suites with incomparable conditions. Producing this  
 75 bundle requires capabilities the Harbor agent benchmark runtime does not natively provide: we  
 76 extend Harbor with sandbox-level instrumentation that records the structural diff of the container file  
 77 system across each trial and streams every outbound network event as it occurs, yielding behavioral  
 78 evidence that no prior skill benchmark exposes.

79 To close the loop between evaluation and adoption, we additionally release a **browser extension**  
 80 that brings SkillTestBench results to the point of discovery. The extension recognizes skill-format  
 81 repositories on GitHub by their SKILL.md structure, queries pre-computed evaluation results, and  
 82 renders a compact utility–safety–cost panel directly on the repository page. A user browsing a  
 83 candidate skill sees, before installing, whether the skill has measurable utility, what permission and  
 84 runtime signals it triggers, and how much overhead it introduces. The same evaluation evidence that  
 85 supports scientific comparison is also surfaced where adoption decisions are actually made.

## 86 Contributions.

- 87 • **A reframing of skill evaluation.** We argue that deployable agent skill packages constitute a  
 88 class of AI artifact whose evaluation requires a generator-style benchmark: skills are inputs,  
 89 not conditions. We formalize what it means to evaluate one as a deployable artifact along  
 90 three deployment-facing axes.
- 91 • **A methodological diagnosis.** We identify and empirically demonstrate two structural failure  
 92 modes of naive skill evaluation—baseline conflation and task-difficulty misalignment—and  
 93 address them with paired execution under matched conditions combined with skill-grounded  
 94 task generation.
- 95 • **An automated, executable framework.** SkillTestBench converts an arbitrary skill pack-  
 96 age into verifier-backed Harbor tasks and reports utility, safety, and cost from a shared  
 97 instrumented execution pipeline, without per-skill manual curation.
- 98 • **Sandbox-level skill instrumentation.** We extend the Harbor agent benchmark runtime with  
 99 two capabilities it does not natively support: structural diffing of the container file system  
 100 across each trial, and real-time logging of all outbound network activity. Together these  
 101 expose runtime evidence—which files a skill writes, where a skill connects—that is essential  
 102 for the safety axis but absent from prior skill benchmarks.
- 103 • **An in-the-wild discovery layer.** A browser extension that detects skill repositories on  
 104 GitHub and surfaces SkillTestBench results inline, turning a research benchmark into a  
 105 signal available at adoption time.
- 106 • **Empirical findings on a curated skill suite.** On [N] skills across [K] categories yielding  
 107 [M] generated tasks, we show that [headline finding 1: e.g., a substantial fraction of public  
 108 skills provide near-zero paired improvement once tasks are evaluated in matched with-  
 109 skill/no-skill pairs], [headline finding 2: e.g., static and runtime risk signals diverge in [Z]%  
 110 of skills], and [headline finding 3: e.g., utility gains and resource overhead cluster into  
 111 deployment-relevant regimes], revealing substantial gaps between the documented claims of  
 112 public skills and their measured deployment value.

## 113 2 Related Work

114 SkillTestBench sits at the intersection of three lines of work: benchmarking LLM agents, evaluating  
 115 tool and procedural augmentation, and security evaluation of agent ecosystems. Table 1 summarizes  
 116 the comparison.

**Benchmarking LLM agents.** A first wave of agent benchmarks evaluated end-to-end agent capability on fixed task suites. AgentBench [Liu et al., 2024] introduced a multi-environment evaluation across eight scenarios; subsequent work emphasized realism (TheAgentCompany [Xu et al., 2025], which embeds agents in a simulated company), reliability ( $\tau$ -bench [Yao et al., 2025], which introduced the pass<sup>k</sup> consistency metric and showed that single-trial success rates substantially overstate practical reliability), and software engineering (SWE-bench [Jimenez et al., 2024]). These benchmarks measure how well an *agent* performs on a fixed set of tasks. The unit of evaluation is the agent; the tasks are the yardstick. SkillTestBench differs at this level: it evaluates a *skill artifact*, and tasks are generated as a function of the skill rather than fixed in advance.

**Tool use and procedural augmentation.** A second line of work evaluates how well language models invoke external capabilities. ToolLLM/ToolBench [Qin et al., 2024] introduced large-scale benchmarking over thousands of real APIs; BFCL [Patil et al., 2025] standardized function-calling evaluation through abstract syntax tree matching and serves as the de facto leaderboard for native function calling. These benchmarks evaluate whether a model *can correctly invoke* a tool given its specification—they treat the tool as a stable interface. Skills, by contrast, are deployable packages bundling documentation, executable code, dependencies, and environment assumptions: their value cannot be reduced to interface correctness, and their risks cannot be assessed without runtime execution. A complementary thread studies procedural augmentation more broadly—retrieval-augmented generation [Lewis et al., 2020], ReAct-style reasoning traces [Yao et al., 2023], cognitive architectures for language agents [Sumers et al., 2024]—but rarely quantifies the deployment impact of any specific augmentation artifact. Recent surveys note this gap explicitly: many augmentation methods exist; very few are benchmarked as artifacts that one would adopt or reject.

**Skills as first-class evaluation artifacts.** The closest prior work is SkillsBench [Li et al., 2026], which also argues that skills should be treated as first-class evaluation artifacts and which, to our knowledge, is the first published benchmark to do so. SkillsBench curates 84 hand-authored tasks across 11 domains, each paired with hand-authored skills, and evaluates seven agent-model configurations under three conditions (no skills, curated skills, self-generated skills) over 7,308 trajectories. Their core findings—that curated skills improve pass rate by 16.2pp on average with high cross-domain variance, that self-generated skills provide no benefit, and that focused 2–3-module skills outperform comprehensive documentation—establish important empirical baselines for the field, and we adopt their paired-execution protocol as a methodological starting point.

SkillTestBench differs from SkillsBench along three axes that correspond to the limitations explicitly noted in their discussion. *First, evaluation object.* SkillsBench fixes a task suite and treats skills as a condition variable: a new skill cannot be evaluated without authors first hand-crafting matching tasks. SkillTestBench treats the skill as the input and synthesizes capability-targeted tasks from its profile, enabling evaluation of arbitrary skills, including the lower-quality, ecosystem-representative skills that SkillsBench’s appendix identifies as out of scope (their benchmark selects from the top quality quartile of a 47,150-skill ecosystem whose mean quality score is 6.2/12). *Second, evaluation axes.* SkillsBench reports utility (pass rate) and discusses cost post-hoc through a Pareto analysis; it does not evaluate permission or runtime risk. SkillTestBench reports utility, safety, and cost jointly from the same execution evidence, motivated by the observation that the same ecosystem dynamics that produce useful skills also produce skills with unvetted runtime behavior. *Third, distribution to users.* SkillsBench is a research benchmark released for community use; its consumers are researchers running evaluations. SkillTestBench additionally surfaces evaluation results to end users at the point of adoption through a browser extension over GitHub, on the premise that the scientific value of a skill evaluation is partially realized through its accessibility at the moment a deployment decision is made.

**Security evaluation of agent ecosystems.** A parallel research community studies the security of tool-augmented agents. InjecAgent [Zhan et al., 2024] introduced a 1,054-case benchmark for indirect prompt-injection attacks and showed that ReAct-prompted GPT-4 is vulnerable 24% of the time; subsequent work demonstrated tool-poisoning attacks [Shi et al., 2025] and covert exfiltration via MCP servers. These works construct adversarial inputs and measure agent robustness; they do not evaluate naturalistic skills under benign conditions. Conversely, capability benchmarks ignore security entirely. SkillTestBench bridges these by including permission and runtime risk as a first-class evaluation axis, derived from static analysis of the skill package combined with dynamic evidence

**Table 1:** Positioning of SkillTestBench relative to representative prior work. “Skill as input” indicates whether the benchmark generates evaluation tasks from arbitrary skill packages, as opposed to fixing a task suite and varying skills as a condition. “Joint axes” indicates whether utility, safety, and cost are reported from the same execution evidence.

|                                   | Eval. object | Skill as input | Joint axes             | In-the-wild surfacing   |
|-----------------------------------|--------------|----------------|------------------------|-------------------------|
| AgentBench [Liu et al., 2024]     | Agent        | –              | –                      | –                       |
| $\tau$ -bench [Yao et al., 2025]  | Agent        | –              | –                      | –                       |
| TheAgentCompany [Xu et al., 2025] | Agent        | –              | –                      | –                       |
| ToolLLM [Qin et al., 2024]        | Tool call    | –              | –                      | –                       |
| BFCL [Patil et al., 2025]         | Tool call    | –              | –                      | –                       |
| InjecAgent [Zhan et al., 2024]    | Attack       | –              | Sec. only              | –                       |
| SkillsBench [Li et al., 2026]     | Skill        | Condition      | Util. only             | –                       |
| <b>SkillTestBench (ours)</b>      | <b>Skill</b> | <b>Input</b>   | <b>Util.+Sec.+Cost</b> | <b>GitHub extension</b> |

from sandboxed execution—network egress, file-system mutations, and trajectory-level behavioral signals—collected during the same containerized run that produces the utility measurement.

### 3 Framework

SkillTestBench evaluates deployable agent skills rather than fixed task suites. It treats the skill package itself—documentation, implementation files, runtime assumptions, and operational risks—as the unit of analysis, and reports three outcomes for each package: utility, safety, and operational cost. Safety combines security analysis with permission-risk assessment.

#### 3.1 Pipeline Overview

Figure 2 sketches the full pipeline. Each collected skill is first normalized into a benchmarkable object, then split into utility and security branches for scheme generation and task construction before rejoining at execution and aggregation. Utility is estimated with paired no-skill and with-skill runs, whereas safety is assessed through static analysis and, when needed, targeted runtime probes.

For the  $j$ -th collected skill, let  $\mathcal{K}_j$  denote the skill package,  $\mathcal{X}_j = (X_j^{\text{util}}, X_j^{\text{sec}})$  the generated utility and security schemes,  $\mathcal{T}_j$  the executable task set,  $\mathcal{E}_j$  the resulting evidence bundles, and  $\mathcal{R}_j$  the final report. With  $\Omega$  denoting scheme generation,  $G$  task construction, Harbor sandboxed execution, and  $\Phi$  aggregation, the end-to-end mapping is

$$\mathcal{K}_j \xrightarrow{\Omega} \mathcal{X}_j \xrightarrow{G} \mathcal{T}_j \xrightarrow{\text{Harbor}} \mathcal{E}_j \xrightarrow{\Phi} \mathcal{R}_j, \quad (1)$$

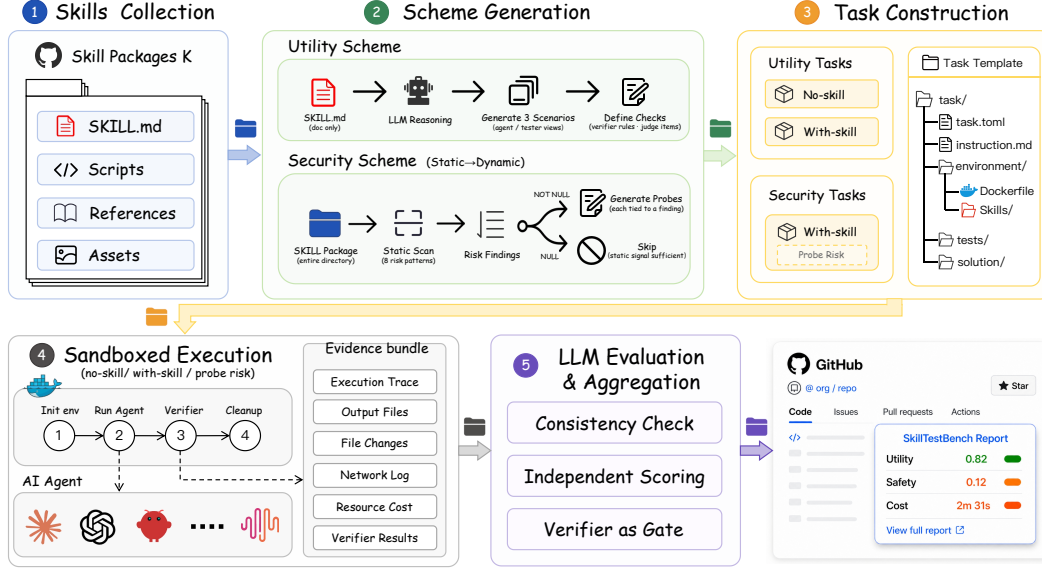
The pipeline has five stages: (i) skill collection, (ii) scheme generation, (iii) task construction, (iv) sandboxed execution and evidence collection, and (v) LLM evaluation and aggregation. Each stage passes explicit artifacts to the next: scheme generation specifies what to test, task construction turns those specifications into Harbor tasks, execution records the resulting behavior, and aggregation converts the evidence into report-level judgments. Every reported signal is therefore traceable to its originating skill, task, and execution record.

#### 3.2 Skill Collection

The pipeline begins by filtering raw external packages into benchmarkable skills and placing each admitted package into a common representation. A candidate is treated as a self-contained directory whose documentation, executable components, and auxiliary materials together define the behavior it may expose to an agent. The  $j$ -th admitted skill is represented as

$$\mathcal{K}_j = (\mathcal{D}_j, \mathcal{F}_j, \mathcal{M}_j), \quad (2)$$

where  $\mathcal{D}_j$  denotes the documentation rooted at `SKILL.md`,  $\mathcal{F}_j$  denotes the implementation bundle, including scripts, templates, references, configuration files, and assets, and  $\mathcal{M}_j$  denotes provenance metadata such as source, owner, category, and package name.



**Figure 2: SkillTestBench pipeline.** A collected skill is translated into utility and security schemes, compiled into paired utility tasks and security probes, executed in isolated sandboxes, and aggregated into utility, safety, and cost signals.

A directory is admitted if and only if it contains a root `SKILL.md`. This criterion follows prevailing skill-package conventions while remaining agnostic to internal implementation style. SkillTestBench therefore assumes neither a fixed API nor any particular programming language or script layout. Those implementation details remain encapsulated in  $\mathcal{F}_j$  and are interpreted only later, when the framework generates scenarios, scans for risks, and exposes the skill during execution.

This stage yields the collected skill set

$$\mathcal{K} = \{\mathcal{K}_j\}_{j=1}^N, \quad (3)$$

along with the provenance records needed to trace every downstream scheme, task, execution bundle, and report back to its source package.

### 3.3 Scheme Generation

Before any Harbor task is instantiated, SkillTestBench derives from each collected skill a utility scheme and a security scheme. The utility branch is based on `SKILL.md`, which we treat as the authoritative description of the skill’s intended capability and usage conditions. The security branch is derived from the full package, because deployment-relevant risks may appear not only in the documentation but also in code, templates, configuration files, dependencies, or other auxiliary assets. Together, the two schemes specify what will be tested, which evidence will be collected, and how that evidence will later be judged.

Formally,

$$\mathcal{X}_j = \Omega(\mathcal{K}_j) = (X_j^{\text{util}}, X_j^{\text{sec}}), \quad (4)$$

where the utility and security branches are generated separately and then passed to a shared task-construction stage.

#### 3.3.1 Utility Scheme

In the current implementation, the generator produces three capability-oriented utility scenarios for each skill,

$$X_j^{\text{util}} = \{x_{j,1}, x_{j,2}, x_{j,3}\}, \quad (5)$$

which provide several views of the claimed capability without making evaluation prohibitively expensive for large skill collections.

Each scenario is represented as

$$x_{j,k} = (I_{j,k}, E_{j,k}, V_{j,k}^{\text{rule}}, V_{j,k}^{\text{judge}}, P_{j,k}), \quad (6)$$

where  $I_{j,k}$  is the agent-facing instruction,  $E_{j,k}$  specifies the execution context and provided materials,  $V_{j,k}^{\text{rule}}$  contains deterministic checks,  $V_{j,k}^{\text{judge}}$  contains semantic evaluation items, and  $P_{j,k}$  specifies the skill-exposure contract used when the skill-enabled variant of the scenario is instantiated.

$P_{j,k}$  is not a target answer. It simply specifies how the skill is made available to the agent in the skill-enabled condition so that availability is handled consistently across scenarios. Only  $I_{j,k}$  and the ordinary task interface are visible to the agent. Execution context, verifier checks, judge items, and exposure metadata remain internal to the benchmark. This keeps evaluation criteria out of the prompt while ensuring that the no-skill and with-skill versions share the same user-facing instruction. The two check sets play different roles:  $V_{j,k}^{\text{rule}}$  is compiled into the structural verifier, whereas  $V_{j,k}^{\text{judge}}$  is reserved for LLM-based aggregation.

### 3.3.2 Security Scheme

The security scheme follows the static-to-dynamic path shown in Figure 2. The full skill package is first scanned to produce a structured risk inventory,

$$\Sigma_j = (F_j^{\text{sec}}, N_j, Q_j), \quad (7)$$

where  $F_j^{\text{sec}} = \{e_{j,1}, \dots, e_{j,m}\}$  is the set of static findings,  $N_j$  summarizes those findings by severity, and  $Q_j \subseteq F_j^{\text{sec}}$  is the subset that requires runtime confirmation. The scan targets deployment-relevant patterns such as prompt injection, credential access, unsafe file or network operations, hidden instructions, and supply-chain risk. The queue  $Q_j$  contains the findings that cannot be resolved from static evidence alone and thus motivate runtime probes.

Dynamic security scenarios are generated only when  $Q_j \neq \emptyset$ :

$$X_j^{\text{sec}} = \begin{cases} \emptyset, & Q_j = \emptyset, \\ \Omega_{\text{sec}}(\mathcal{K}_j, \Sigma_j), & Q_j \neq \emptyset. \end{cases} \quad (8)$$

Each generated security scenario carries forward both the finding under test and the evidence criteria used later for runtime confirmation. When  $Q_j = \emptyset$ , the security branch remains purely static.

### 3.4 Task Construction

The resulting schemes are then compiled into an executable task set  $\mathcal{T}_j$ . As in Figure 2, utility scenarios become matched task pairs, whereas dynamic security findings become single security probes with the skill available. Because both task families share the same overall structure, they can be executed and aggregated within a common pipeline.

#### 3.4.1 Shared Task Template

Every generated task contains the same basic elements: an agent-facing instruction, an execution environment, verification logic, and reference materials. This shared template fixes the core interface while leaving the content of individual scenarios free to vary. When a scenario requires skill access, the environment includes a designated slot through which the evaluated skill can be exposed.

By design, task semantics remain distinct from execution mechanics. Utility tasks and security probes may differ in their instructions, inputs, and checks, but they pass through the same evaluation interface.

#### 3.4.2 Utility Task Pairing

For each utility scenario  $x_{j,k} \in X_j^{\text{util}}$ , the construction map produces a matched task pair

$$G_{\text{util}} : x_{j,k} \mapsto (\tau_{j,k}^{\text{wo}}, \tau_{j,k}^{\text{wi}}), \quad (9)$$

where  $\tau_{j,k}^{\text{wo}}$  is the no-skill task and  $\tau_{j,k}^{\text{wi}}$  is the with-skill task. The two tasks share the same instruction, inputs, environment, and evaluation criteria. In the with-skill condition, the exposure contract makes the evaluated skill available to the agent:

$$\tau_{j,k}^{\text{wi}} = \tau_{j,k}^{\text{wo}} \cup \{\text{expose}(\mathcal{K}_j, P_{j,k})\}. \quad (10)$$

The no-skill counterpart keeps the same surrounding interface but does not expose the skill.

Skill availability is therefore the only intended difference between the two conditions. Differences in outputs, behavior, verifier outcomes, time, and token usage can accordingly be interpreted as effects of the skill setting rather than artifacts of separately constructed tasks.

### 3.4.3 Security-Probing Tasks

For each dynamic security scenario  $x_{j,\ell}^{\text{sec}} \in X_j^{\text{sec}}$ , the construction map yields a single security probe with the skill available

$$G_{\text{sec}} : x_{j,\ell}^{\text{sec}} \mapsto \tau_{j,\ell}^{\text{run}}. \quad (11)$$

A no-skill counterpart is unnecessary because the goal here is not counterfactual utility estimation, but runtime confirmation of a previously identified risk hypothesis. The probe uses the same shared template and carries forward the evidence criteria associated with that finding.

The complete task set for skill  $\mathcal{K}_j$  is

$$\mathcal{T}_j = \bigcup_{k=1}^{|X_j^{\text{util}}|} \{\tau_{j,k}^{\text{wo}}, \tau_{j,k}^{\text{wi}}\} \cup \bigcup_{\ell=1}^{|X_j^{\text{sec}}|} \{\tau_{j,\ell}^{\text{run}}\}. \quad (12)$$

In the current implementation,  $|X_j^{\text{util}}| = 3$ ; if no dynamic security scenarios are generated, the second union is empty. All tasks then move to a shared execution stage.

## 3.5 Sandboxed Execution and Evidence Collection

After construction, the tasks in  $\mathcal{T}_j$  are executed in isolated Harbor environments, producing trial-level evidence for utility, safety, and cost reporting. Although utility pairs and security probes originate from different branches, they are executed under the same evaluation and logging setup.

### 3.5.1 Execution Protocol

Each task is run with a chosen agent inside this shared evaluation environment. SkillTestBench does not assume any particular agent architecture; it requires only that the agent can follow the instruction and operate within the sandbox. Each trial proceeds through setup, agent interaction, verification, and cleanup.

At execution time, we represent a job as

$$\mathcal{J}_j = (\mathcal{T}_j, A, C, V, R), \quad (13)$$

where  $\mathcal{T}_j$  is the task set for skill  $\mathcal{K}_j$ ,  $A$  is the agent backend, and  $C$ ,  $V$ , and  $R$  summarize the execution environment, evaluation setup, and run controls. The same job structure is used for no-skill utility tasks, with-skill utility tasks, and security probes.

### 3.5.2 Evidence Bundle

Each executed trial  $i$  yields an evidence bundle

$$\mathcal{E}_i = (\Gamma_i, O_i, \Delta F_i, N_i, C_i, V_i), \quad (14)$$

where  $\Gamma_i$  is the execution trace,  $O_i$  is the set of produced artifacts,  $\Delta F_i$  records file-system changes,  $N_i$  records network activity,  $C_i$  records resource usage, and  $V_i$  is the verifier outcome. The set of bundles associated with skill  $\mathcal{K}_j$  is denoted by  $\mathcal{E}_j$  in the stage mapping above.

The bundle is richer than any single downstream metric because its components support different kinds of judgment. Trajectories and artifacts support semantic evaluation, file and network logs

support safety analysis, resource counters support cost reporting, and verifier results support structural gating. At the skill level, the final report draws on multiple such bundles, including paired utility runs and, when needed, separate security probes, all collected under the same runtime controls and logging schema.

### 3.5.3 Common-Instrumentation Provenance

Shared provenance matters because agent behavior is stochastic. If semantic utility, resource usage, and runtime risk were measured under different logging regimes, the connection between an output and its operational evidence would be lost. For each utility scenario  $x_{j,k}$ , the framework therefore records a matched pair of bundles

$$(\mathcal{E}_{j,k}^{\text{wo}}, \mathcal{E}_{j,k}^{\text{wi}}), \quad (15)$$

and for each security probe it records a single risk-oriented bundle  $\mathcal{E}_{j,\ell}^{\text{run}}$ . These records are carried forward unchanged. Aggregation may summarize them for judgment, but the underlying trial-level evidence remains available for audit, reruns, and post-hoc analysis. Comparability at the skill level therefore comes from shared task design and instrumentation, not from forcing every signal to arise from a single trial.

## 3.6 LLM Evaluation and Aggregation

Aggregation converts trial-level evidence into report-level judgments. On the utility branch, it first checks whether a pair is suitable for comparison, then scores the no-skill and with-skill runs independently against the same semantic criteria, and uses the verifier only as a structural gate. On the security branch, it asks whether runtime evidence confirms the queued static findings. Resource counters are read directly from the same bundles, without a separate judging step.

### 3.6.1 Consistency Check

Before semantic scoring, each utility pair is checked for comparability using its execution evidence. We write  $\text{judgeable}(\mathcal{E}) \in \{0, 1\}$  when a trial reaches semantic evaluation without setup failure or fatal verifier error. We write  $\text{access}(\Gamma, P) \in \{0, 1\}$  when the trajectory  $\Gamma$  makes use of the skill exposure described by  $P$ . A pair is considered valid when both trials are judgeable:

$$\text{valid}_{j,k} = \mathbf{1}[\text{judgeable}(\mathcal{E}_{j,k}^{\text{wi}}) = 1 \wedge \text{judgeable}(\mathcal{E}_{j,k}^{\text{wo}}) = 1]. \quad (16)$$

The access indicators  $\text{access}(\Gamma_{j,k}^{\text{wi}}, P_{j,k})$  and  $\text{access}(\Gamma_{j,k}^{\text{wo}}, P_{j,k})$  are kept as diagnostics rather than used as filtering criteria. A with-skill run that never uses the exposed skill still counts toward realized deployment utility: the skill was available, but the agent did not use it successfully. Only structurally incomparable pairs are excluded from aggregate utility estimation.

### 3.6.2 Independent Capability Scoring

For each valid utility scenario, the LLM evaluator receives the paired evidence bundles

$$(\mathcal{E}_{j,k}^{\text{wo}}, \mathcal{E}_{j,k}^{\text{wi}})$$

together with the scenario’s semantic evaluation items  $V_{j,k}^{\text{judge}}$ . For each item  $\ell$ , the evaluator assigns separate binary pass indicators to the no-skill and with-skill runs:

$$\Phi_{\text{util}}(\mathcal{E}_{j,k}^{\text{wo}}, \mathcal{E}_{j,k}^{\text{wi}}, V_{j,k}^{\text{judge}}) = \{(s_{\ell}^{\text{wo}}, s_{\ell}^{\text{wi}})\}_{\ell=1}^{|V_{j,k}^{\text{judge}}|}, \quad s_{\ell}^{(\cdot)} \in \{0, 1\}. \quad (17)$$

Rather than asking the evaluator to compare the two runs directly, we ask whether each run independently satisfies each capability criterion. That choice preserves the paired design while allowing a strong no-skill baseline to receive full credit when it already meets the criterion.

### 3.6.3 Verifier Gating

Verifier outcomes serve as structural gates rather than full measures of utility. They flag missing artifacts, malformed outputs, runtime exceptions, and failed rule-based checks. Semantic capability still comes from the LLM-scored item set, while resource usage is read directly from the evidence bundle. This keeps structural correctness, semantic success, and operational cost conceptually distinct.

### 3.6.4 Security Confirmation and Cost Aggregation

For each security probe, the evaluator receives the risk-oriented evidence bundle  $\mathcal{E}_{j,\ell}^{\text{run}}$  together with the expected-evidence description inherited from the corresponding static finding. The resulting security judgment is

$$\Phi_{\text{sec}}(\mathcal{E}_{j,\ell}^{\text{run}}, \text{expected}_{j,\ell}) = (r_{j,\ell}, S_{j,\ell}), \quad r_{j,\ell} \in \{0, 1\}, \quad (18)$$

where  $r_{j,\ell}$  indicates whether the expected runtime signal is observed and  $S_{j,\ell}$  records the supporting evidence summary.

Cost requires no additional judge. Wall-clock time, token usage, and related resource counters are aggregated directly from the same executions used for utility and security assessment. The final report for skill  $\mathcal{K}_j$  is

$$\mathcal{R}_j = (M_j^{\text{util}}, M_j^{\text{sec}}, M_j^{\text{cost}}), \quad (19)$$

where  $M_j^{\text{util}}$  summarizes valid paired utility evidence,  $M_j^{\text{sec}}$  summarizes static and dynamically confirmed risk signals, and  $M_j^{\text{cost}}$  summarizes resource consumption from the same execution records. Invalid utility pairs and confirmed risk evidence are retained as diagnostic fields rather than discarded during aggregation. This report is the final output of the framework; the next section specifies how its components are quantified.

## 4 Evaluation Dimensions and Metrics

The previous section produced, for each skill  $\mathcal{K}_j$ , an evidence bundle record  $\mathcal{E}_j$  together with a structured report  $\mathcal{R}_j = (M_j^{\text{util}}, M_j^{\text{sec}}, M_j^{\text{cost}})$ . This section specifies how each component is reduced into a reported signal. Throughout this section, *wi* denotes the with-skill condition and *wo* denotes the matched no-skill condition.

### 4.1 Overview

We keep the three signals on independent reporting axes. Utility answers whether the skill helps; safety answers whether it is safe to enable; cost answers how enabling the skill changes time and token usage. Collapsing them into one score would hide practically important disagreements. For example, a skill may improve task completion while introducing credential risk, or it may be safe and useful while substantially increasing wall-clock time. Such cases should not receive the same numeric score as one that is balanced across all three axes.

Table 2 summarizes the metric groups, their evidence sources, and their reported fields.

**Table 2:** Evaluation dimensions in SkillTestBench. The three groups draw on overlapping evidence channels but produce separate report fields.

| Group   | Question                                | Evidence                                                          | Reported fields                               |
|---------|-----------------------------------------|-------------------------------------------------------------------|-----------------------------------------------|
| Utility | Does the skill improve task completion? | Paired wi/wo runs, judge items, verifier gate, skill-access trace | raw_delta, uplift, ceiling_flag, tie_flag     |
| Safety  | Is the skill safe to enable?            | Static findings, dynamic probes, file-system and network logs     | security_score, per-finding deduction, status |
| Cost    | What overhead does the skill introduce? | Wall-clock time and effective input tokens from paired runs       | $\Delta t, \Delta \tilde{q}$                  |

### 4.2 Utility: Attributable Marginal Task Benefit

Utility measures the marginal task benefit of enabling a skill relative to its matched no-skill condition. For each utility scenario  $x_{j,k}$ , the framework records a paired bundle  $(\mathcal{E}_{j,k}^{\text{wo}}, \mathcal{E}_{j,k}^{\text{wi}})$  obtained from two runs that share the same instruction, environment, agent backend, task instance, and run controls; when supported by the agent harness, the random seed is also fixed. This paired design reduces task-construction confounding and supports attribution to skill availability.

**Attribution validity.** Before scoring, each scenario undergoes an attribution check. Let  $\text{access}(\Gamma, P_{j,k}) \in \{0, 1\}$  indicate whether the trajectory  $\Gamma$  accesses the skill exposure described by the scenario’s exposure contract  $P_{j,k}$ . Here, *access* means access to the evaluated skill package through the controlled exposure interface, not the use of independently generated code or external knowledge. A paired scenario is attribution-valid only when

$$\text{valid}_{j,k} = \text{access}(\Gamma_{j,k}^{wi}, P_{j,k}) = 1 \wedge \text{access}(\Gamma_{j,k}^{wo}, P_{j,k}) = 0. \quad (20)$$

The with-skill run must access the evaluated skill through the controlled exposure interface, and the no-skill run must not access the evaluated package or leaked skill artifacts. A with-skill run that never invokes the skill, or a no-skill run that reaches into the evaluated skill package or its leaked artifacts, breaks the counterfactual contrast and makes the resulting outcome unattributable to the skill itself. Importantly, a no-skill run that solves the task through independently generated code is treated as baseline capability and remains in the pool, whereas access to leaked skill artifacts invalidates the pair. This filtering estimates attributable utility rather than natural adoption behavior. Access failures are reported as adoption diagnostics rather than silently discarded; the invalid-scenario count and access-pattern summaries are surfaced together with  $U(\mathcal{K}_j)$  so that low invocation rates remain visible.

**Per-scenario benefit.** Let  $u_{j,k}^{wi}, u_{j,k}^{wo} \in [0, 1]$  denote the with-skill and no-skill pass rates over the scenario’s semantic items, computed as in Section 3.6. We report two complementary per-scenario quantities:

$$\Delta_{j,k}^{\text{raw}} = u_{j,k}^{wi} - u_{j,k}^{wo}, \quad \Delta_{j,k}^{\text{uplift}} = \text{clamp}\left(\frac{u_{j,k}^{wi} - u_{j,k}^{wo}}{\max(1 - u_{j,k}^{wo}, \varepsilon)}, 0, 1\right). \quad (21)$$

These two quantities serve different roles.  $\Delta_{j,k}^{\text{raw}}$  records the absolute pass-rate change.  $\Delta_{j,k}^{\text{uplift}}$  normalizes this change by the no-skill failure margin, measuring how much remaining baseline error is closed by the skill. The asymmetric denominator is deliberate: when  $u_{j,k}^{wo}$  is already close to one, the headroom available to the skill is small by construction, and a denominator proportional to  $u_{j,k}^{wo}$  would penalize ceiling scenarios unfairly. Negative uplift values are clipped to zero in the non-negative utility score  $U(\mathcal{K}_j)$ , while signed raw deltas remain reported.

**Ceiling scenarios.** When  $u_{j,k}^{wo} = 1$ , pass-rate improvement is structurally unobservable because the baseline already satisfies all semantic items. The framework records a per-scenario flag  $\text{ceiling}_{j,k} = \mathbf{1}[u_{j,k}^{wo} = 1]$  to mark that the scenario carries no semantic headroom for the current model. Resource changes in such scenarios are reported on the cost axis (Section 4.4).

**Skill-level aggregation.** Let  $\mathcal{V}_j$  be the set of attribution-valid scenarios admitted by Equation (20). The skill-level utility signal is

$$U(\mathcal{K}_j) = \frac{1}{|\mathcal{V}_j|} \sum_{(j,k) \in \mathcal{V}_j} \Delta_{j,k}^{\text{uplift}}, \quad \Delta^{\text{raw}}(\mathcal{K}_j) = \frac{1}{|\mathcal{V}_j|} \sum_{(j,k) \in \mathcal{V}_j} \Delta_{j,k}^{\text{raw}}. \quad (22)$$

$\Delta^{\text{raw}}(\mathcal{K}_j)$  is not used as the primary utility score, but is reported alongside  $U(\mathcal{K}_j)$  to expose regressions and ceiling artifacts. We also report two diagnostic flags.  $\text{ceiling\_flag}(\mathcal{K}_j)$  is true when every valid scenario satisfies  $\text{ceiling}_{j,k} = 1$ , indicating that pass-rate benefit is unobservable because the baseline already saturates the semantic items.  $\text{tie\_flag}(\mathcal{K}_j)$  is true when  $u_{j,k}^{wi} = u_{j,k}^{wo}$  holds for every valid scenario, regardless of whether they are at the ceiling. Both flags are diagnostic and do not change the utility score. If  $\mathcal{V}_j = \emptyset$ , the skill receives  $U(\mathcal{K}_j) = \text{null}$  together with a diagnostic note, since the paired design did not produce a valid counterfactual. Safety does not rescale  $U(\mathcal{K}_j)$ ; unsafe but useful skills are exposed as cross-axis disagreement rather than hidden by scalar collapse.

### 4.3 Safety: Static–Dynamic Risk Assessment

A skill can complete tasks correctly while still introducing deployment risk, so the safety axis is computed independently of utility. The score combines two stages of evidence: static analysis of the skill package estimates whether each risk pattern is present in the documentation, code, configuration, or auxiliary files, while dynamic probes estimate whether the risk can be triggered at runtime. The product of these two confidences scales a severity-derived penalty, and the accumulated deduction is subtracted from a baseline safety score of 100.

**Static findings and existence confidence.** The scanner introduced in Section 3.3 emits a finding inventory  $F_j^{\text{sec}} = \{e_{j,1}, \dots, e_{j,m}\}$ . Each finding  $e_{j,\ell}$  carries a severity level  $\sigma_{j,\ell} \in \{H, M, L\}$  and an existence confidence  $c_{j,\ell}^{\text{exist}} \in [0, 1]$ . Severity captures the inherent gravity of the risk class and is mapped to a fixed base penalty by  $w(\sigma)$ , with  $w(H) = 12$ ,  $w(M) = 6$ ,  $w(L) = 3$ . Existence confidence estimates how likely the risk is to be present given the static evidence. It is high when the evidence is direct and unambiguous, for example, unsanitized input flowing into an execution sink, and lower when the finding rests on indirect inference or admits a benign explanation. When measured fields are unavailable, we use a default confidence of 0.6 and mark it in the per-finding detail record.

**Dynamic exploitability.** Findings flagged for runtime confirmation are routed through the dynamic queue  $Q_j$  of Section 3.3. For each such finding, the security probe yields an exploitability score  $c_{j,\ell}^{\text{exploit}} \in [0, 1]$  assigned by a security judge that combines three sources: deterministic verifier signals over the trial bundle (network egress, file writes, credential access, and similar observable side-effects), the LLM judge’s semantic reading of the trajectory and output, and the expected-evidence description inherited from the static finding. The combined judgment is summarized by a discrete trigger verdict whose mapping to  $c^{\text{exploit}}$  is shown in Table 3. The `agent_refused` case receives a moderate value rather than zero because the risky pattern remains in the package and may transfer to agents with weaker safety policies. For findings without a valid dynamic probe, we use the same default value,  $c^{\text{exploit}} = 0.6$ , and mark the source in the detail record. Findings with default confidence values are flagged separately in the diagnostic record.

**Table 3:** Mapping from trigger verdict to exploitability score. The verdict is produced by the security judge using verifier signals, the execution trajectory, and the expected-evidence description from the corresponding static finding.

| Trigger verdict           | $c^{\text{exploit}}$ | Interpretation                                                     |
|---------------------------|----------------------|--------------------------------------------------------------------|
| confirmed                 | 0.7–1.0              | End-to-end attack path executed with observed evidence.            |
| agent_refused             | 0.5                  | Agent declined; risk present but blocked by current safety policy. |
| path_exists_not_triggered | 0.2–0.4              | Path is reachable but not exercised on this run.                   |
| likely_false_positive     | 0.1–0.2              | Trajectory shows no engagement with the predicted path.            |

**Score and reported detail.** Each finding contributes a deduction proportional to the joint confidence that the risk is both present and exploitable:

$$d_{j,\ell} = w(\sigma_{j,\ell}) \cdot c_{j,\ell}^{\text{exist}} \cdot c_{j,\ell}^{\text{exploit}}, \quad S(\mathcal{K}_j) = \max\left(10, 100 - \sum_{\ell=1}^m d_{j,\ell}\right). \quad (23)$$

The lower bound at ten caps the cumulative deduction and keeps the score within an interpretable range; a skill that hits the floor is reported as `Risky` regardless. The published score is mapped to a three-level status label using a single threshold  $\theta_s$ :  $S(\mathcal{K}_j) = 100$  is reported as `Pass`,  $\theta_s \leq S(\mathcal{K}_j) < 100$  as `Caution`, and  $S(\mathcal{K}_j) < \theta_s$  as `Risky`. We use  $\theta_s = 80$  following the convention established in SkillTester [Wang et al., 2026]. The label summarizes the score, while the per-finding record retains  $(\sigma, c^{\text{exist}}, c^{\text{exploit}}, \text{verdict}, d)$  for audit.

#### 4.4 Cost and Reporting

Cost measures the operational overhead of enabling the skill relative to the matched no-skill condition, using resource counters from the paired execution bundles. We do not normalize these quantities into a  $[0, 1]$  score. Time and token overhead are reported in their native units. Cost deltas are descriptive rather than normative, since acceptable overhead depends on the deployment budget.

**Per-pair deltas.** For each attribution-valid utility scenario, two differences are recorded:

$$\Delta t_{j,k} = t_{j,k}^{wi} - t_{j,k}^{wo}, \quad \Delta \tilde{q}_{j,k} = \tilde{q}_{j,k}^{wi} - \tilde{q}_{j,k}^{wo}, \quad (24)$$

where  $t$  is wall-clock time and  $\tilde{q} = q_{\text{input}} - q_{\text{cache}}$  is the effective input-token count. A positive value indicates overhead introduced by the skill, while a negative value indicates savings. We use effective tokens rather than raw input tokens because cached prefixes are not recomputed by the model and their volume depends on conversation history and orchestrator configuration in ways that are not attributable to the skill itself; the effective count better reflects the fresh computation associated with the skill condition. Skill-level overhead is the average of these per-pair deltas over  $\mathcal{V}_j$ , reported as the pair  $\Delta M^{\text{cost}}(\mathcal{K}_j) = (\Delta t, \Delta \tilde{q})$ . Cost averages are computed over the same attribution-valid set  $\mathcal{V}_j$  used for utility, so utility and cost share a consistent paired comparison. Retry and recoverable-failure counts from the trial trajectories are retained as diagnostic stability signals rather than primary cost metrics.

**Joint reporting.** The skill-level report is the structured tuple

$$\text{Report}(\mathcal{K}_j) = (U(\mathcal{K}_j), S(\mathcal{K}_j), \Delta M^{\text{cost}}(\mathcal{K}_j), \text{diag}(\mathcal{K}_j)), \quad (25)$$

where  $\text{diag}(\mathcal{K}_j)$  collects the diagnostic fields produced along the way: invalid pairs with their access patterns, ceiling and tie flags, per-finding security details with default markers, retry and recoverable-failure counts, and raw resource counters before differencing. When utility cannot distinguish skills under the same safety status, the cost axis serves as a secondary signal in the reporting layer, without entering  $U(\mathcal{K}_j)$  or  $S(\mathcal{K}_j)$ .

## 5 Citations, figures, tables, references

These instructions apply to everyone.

### 5.1 Citations within the text

The `natbib` package will be loaded for you by default. Citations may be author/year or numeric, as long as you maintain internal consistency. As to the format of the references themselves, any style is acceptable as long as it is used consistently.

The documentation for `natbib` may be found at

`http://mirrors.ctan.org/macros/latex/contrib/natbib/natnotes.pdf`

Of note is the command `\citet`, which produces citations appropriate for use in inline text. For example,

`\citet{hasselmo}` investigated\dots

produces

Hasselmo, et al. (1995) investigated...

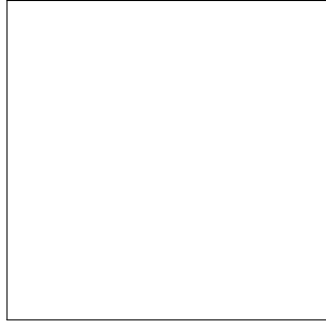
If you wish to load the `natbib` package with options, you may add the following before loading the `neurips_2026` package:

`\PassOptionsToPackage{options}{natbib}`

If `natbib` clashes with another package you load, you can add the optional argument `nonatbib` when loading the style file:

`\usepackage[nonatbib]{neurips_2026}`

As submission is double blind, refer to your own published work in the third person. That is, use “In the previous work of Jones et al. [4],” not “In our previous work [4].” If you cite your other papers that are not widely available (e.g., a journal paper under review), use anonymous author names in the citation, e.g., an author of the form “A. Anonymous” and include a copy of the anonymized paper in the supplementary material.



**Figure 3:** Sample figure caption. Explain what the figure shows and add a key take-away message to the caption.

**Table 4:** Sample table caption. Explain what the table shows and add a key take-away message to the caption.

| Part     |                 |                        |
|----------|-----------------|------------------------|
| Name     | Description     | Size ( $\mu\text{m}$ ) |
| Dendrite | Input terminal  | $\approx 100$          |
| Axon     | Output terminal | $\approx 10$           |
| Soma     | Cell body       | up to $10^6$           |

## 5.2 Footnotes

Footnotes should be used sparingly. If you do require a footnote, indicate footnotes with a number<sup>1</sup> in the text. Place the footnotes at the bottom of the page on which they appear. Precede the footnote with a horizontal rule of 2 inches (12 picas).

Note that footnotes are properly typeset *after* punctuation marks.<sup>2</sup>

## 5.3 Figures

All artwork must be neat, clean, and legible. Lines should be dark enough for reproduction purposes. The figure number and caption always appear after the figure. Place one line space before the figure caption and one line space after the figure. The figure caption should be lower case (except for the first word and proper nouns); figures are numbered consecutively.

You may use color figures. However, it is best for the figure captions and the paper body to be legible if the paper is printed in either black/white or in color.

## 5.4 Tables

All tables must be centered, neat, clean, and legible. The table number and title always appear before the table. See Table 4.

Place one line space before the table title, one line space after the table title, and one line space after the table. The table title must be lower case (except for the first word and proper nouns); tables are numbered consecutively.

Note that publication-quality tables *do not contain vertical rules*. We strongly suggest the use of the booktabs package, which allows for typesetting high-quality, professional tables:

<https://www.ctan.org/pkg/booktabs>

This package was used to typeset Table 4.

---

<sup>1</sup>Sample of the first footnote.

<sup>2</sup>As in this example.

## 514 5.5 Math

515 Note that display math in bare TeX commands will not create correct line numbers for sub-  
516 mission. Please use LaTeX (or AMSTeX) commands for unnumbered display math. (You  
517 really shouldn't be using \$\$ anyway; see <https://tex.stackexchange.com/questions/503/why-is-preferable-to> and [https://tex.stackexchange.com/questions/40492/](https://tex.stackexchange.com/questions/40492/what-are-the-differences-between-align-equation-and-displaymath)  
518 [what-are-the-differences-between-align-equation-and-displaymath](https://tex.stackexchange.com/questions/40492/what-are-the-differences-between-align-equation-and-displaymath) for more infor-  
519 mation.)  
520

## 521 5.6 Final instructions

522 Do not change any aspects of the formatting parameters in the style files. In particular, do not modify  
523 the width or length of the rectangle the text should fit into, and do not change font sizes. Please note  
524 that pages should be numbered.

## 525 6 Preparing PDF files

526 Please prepare submission files with paper size "US Letter," and not, for example, "A4."

527 Fonts were the main cause of problems in the past years. Your PDF file must only contain Type 1 or  
528 Embedded TrueType fonts. Here are a few instructions to achieve this.

- 529 • You should directly generate PDF files using `pdflatex`.
- 530 • You can check which fonts a PDF file uses. In Acrobat Reader, select the menu  
531 Files>Document Properties>Fonts and select Show All Fonts. You can also use the program  
532 `pdf fonts` which comes with `xpdf` and is available out-of-the-box on most Linux machines.
- 533 • `xfig` "patterned" shapes are implemented with bitmap fonts. Use "solid" shapes instead.
- 534 • The `\bbold` package almost always uses bitmap fonts. You should use the equivalent AMS  
535 Fonts:

536 `\usepackage{amsfonts}`

537 followed by, e.g., `\mathbb{R}`, `\mathbb{N}`, or `\mathbb{C}` for  $\mathbb{R}$ ,  $\mathbb{N}$  or  $\mathbb{C}$ . You can also  
538 use the following workaround for reals, natural and complex:

```
539 \newcommand{\RR}{\mathbb{R}} %real numbers
540 \newcommand{\Nat}{\mathbb{N}} %natural numbers
541 \newcommand{\CC}{\mathbb{C}} %complex numbers
```

542 Note that `amsfonts` is automatically loaded by the `amssymb` package.

543 If your file contains type 3 fonts or non embedded TrueType fonts, we will ask you to fix it.

### 544 6.1 Margins in L<sup>A</sup>T<sub>E</sub>X

545 Most of the margin problems come from figures positioned by hand using `\special` or other  
546 commands. We suggest using the command `\includegraphics` from the `graphicx` package.  
547 Always specify the figure width as a multiple of the line width as in the example below:

```
548 \usepackage[pdftex]{graphicx} ...
549 \includegraphics[width=0.8\linewidth]{myfile.pdf}
```

550 See Section 4.4 in the graphics bundle documentation ([http://mirrors.ctan.org/macros/](http://mirrors.ctan.org/macros/latex/required/graphics/grfguide.pdf)  
551 [latex/required/graphics/grfguide.pdf](http://mirrors.ctan.org/macros/latex/required/graphics/grfguide.pdf))

552 A number of width problems arise when L<sup>A</sup>T<sub>E</sub>X cannot properly hyphenate a line. Please give LaTeX  
553 hyphenation hints using the `\-` command when necessary.

## 554 **References**

555 References follow the acknowledgments in the camera-ready paper. Use unnumbered first-level  
556 heading for the references. Any choice of citation style is acceptable as long as you are consistent. It  
557 is permissible to reduce the font size to small (9 point) when listing the references. Note that the  
558 Reference section does not count towards the page limit.

559 [1] Alexander, J.A. & Mozer, M.C. (1995) Template-based algorithms for connectionist rule extraction. In  
560 G. Tesauero, D.S. Touretzky and T.K. Leen (eds.), *Advances in Neural Information Processing Systems 7*, pp.  
561 609–616. Cambridge, MA: MIT Press.

562 [2] Bower, J.M. & Beeman, D. (1995) *The Book of GENESIS: Exploring Realistic Neural Models with the*  
563 *GENeral NEural Simulation System*. New York: TELOS/Springer-Verlag.

564 [3] Hasselmo, M.E., Schnell, E. & Barkai, E. (1995) Dynamics of learning and recall at excitatory recurrent  
565 synapses and cholinergic modulation in rat hippocampal region CA3. *Journal of Neuroscience* **15**(7):5249-5262.

## 566 **A Technical appendices and supplementary material**

567 Technical appendices with additional results, figures, graphs, and proofs may be submitted with the  
568 paper submission before the full submission deadline (see above). You can upload a ZIP file for  
569 videos or code, but do not upload a separate PDF file for the appendix. There is no page limit for the  
570 technical appendices.

571 Note: Think of the appendix as “optional reading” for reviewers. The paper must be able to stand  
572 alone without the appendix; for example, adding critical experiments that support the main claims to  
573 an appendix is inappropriate.

## NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

**The checklist answers are an integral part of your paper submission.** They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

### 1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

### 2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

### 3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

### 4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

## 5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

## 6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

## 7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

## 8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- 778           • The paper should disclose whether the full research project required more compute  
779           than the experiments reported in the paper (e.g., preliminary or failed experiments that  
780           didn't make it into the paper).

781 **9. Code of ethics**

782 Question: Does the research conducted in the paper conform, in every respect, with the  
783 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

784 Answer: **[TODO]**

785 Justification: **[TODO]**

786 Guidelines:

- 787           • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of  
788           Ethics.  
789           • If the authors answer [No], they should explain the special circumstances that require a  
790           deviation from the Code of Ethics.  
791           • The authors should make sure to preserve anonymity (e.g., if there is a special consid-  
792           eration due to laws or regulations in their jurisdiction).

793 **10. Broader impacts**

794 Question: Does the paper discuss both potential positive societal impacts and negative  
795 societal impacts of the work performed?

796 Answer: **[TODO]**

797 Justification: **[TODO]**

798 Guidelines:

- 799           • The answer [N/A] means that there is no societal impact of the work performed.  
800           • If the authors answer [N/A] or [No], they should explain why their work has no societal  
801           impact or why the paper does not address societal impact.  
802           • Examples of negative societal impacts include potential malicious or unintended uses  
803           (e.g., disinformation, generating fake profiles, surveillance), fairness considerations  
804           (e.g., deployment of technologies that could make decisions that unfairly impact specific  
805           groups), privacy considerations, and security considerations.  
806           • The conference expects that many papers will be foundational research and not tied  
807           to particular applications, let alone deployments. However, if there is a direct path to  
808           any negative applications, the authors should point it out. For example, it is legitimate  
809           to point out that an improvement in the quality of generative models could be used to  
810           generate Deepfakes for disinformation. On the other hand, it is not needed to point out  
811           that a generic algorithm for optimizing neural networks could enable people to train  
812           models that generate Deepfakes faster.  
813           • The authors should consider possible harms that could arise when the technology is  
814           being used as intended and functioning correctly, harms that could arise when the  
815           technology is being used as intended but gives incorrect results, and harms following  
816           from (intentional or unintentional) misuse of the technology.  
817           • If there are negative societal impacts, the authors could also discuss possible mitigation  
818           strategies (e.g., gated release of models, providing defenses in addition to attacks,  
819           mechanisms for monitoring misuse, mechanisms to monitor how a system learns from  
820           feedback over time, improving the efficiency and accessibility of ML).

821 **11. Safeguards**

822 Question: Does the paper describe safeguards that have been put in place for responsible  
823 release of data or models that have a high risk for misuse (e.g., pre-trained language models,  
824 image generators, or scraped datasets)?

825 Answer: **[TODO]**

826 Justification: **[TODO]**

827 Guidelines:

- 828           • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

## 12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, [paperswithcode.com/datasets](https://paperswithcode.com/datasets) has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

## 13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

## 14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

880 Guidelines:

881 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

882 with human subjects.

883 • Including this information in the supplemental material is fine, but if the main contribu-

884 tion of the paper involves human subjects, then as much detail as possible should be

885 included in the main paper.

886 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,

887 or other labor should be paid at least the minimum wage in the country of the data

888 collector.

## 889 15. Institutional review board (IRB) approvals or equivalent for research with human

## 890 subjects

891 Question: Does the paper describe potential risks incurred by study participants, whether

892 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

893 approvals (or an equivalent approval/review based on the requirements of your country or

894 institution) were obtained?

895 Answer: **[TODO]**

896 Justification: **[TODO]**

897 Guidelines:

- 898 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 899 with human subjects.
- 900 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 901 may be required for any human subjects research. If you obtained IRB approval, you
- 902 should clearly state this in the paper.
- 903 • We recognize that the procedures for this may vary significantly between institutions
- 904 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
- 905 guidelines for their institution.
- 906 • For initial submissions, do not include any information that would break anonymity (if
- 907 applicable), such as the institution conducting the review.

## 908 16. Declaration of LLM usage

909 Question: Does the paper describe the usage of LLMs if it is an important, original, or

910 non-standard component of the core methods in this research? Note that if the LLM is used

911 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

912 scientific rigor, or originality of the research, declaration is not required.

913 Answer: **[TODO]**

914 Justification: **[TODO]**

915 Guidelines:

- 916 • The answer [N/A] means that the core method development in this research does not
- 917 involve LLMs as any important, original, or non-standard components.
- 918 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
- 919 be described.

## 920 References

- 921 Anthropic. Equipping agents for the real world with Agent Skills. Anthropic En-
- 922 gineering Blog, October 2025. URL [https://www.anthropic.com/engineering/](https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills)
- 923 [equipping-agents-for-the-real-world-with-agent-skills](https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills). Released as an open stan-
- 924 dard in December 2025; specification at <https://agentskills.io>.
- 925 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R.
- 926 Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The*
- 927 *Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL [https://](https://openreview.net/forum?id=VTF8yNQm66)
- 928 [openreview.net/forum?id=VTF8yNQm66](https://openreview.net/forum?id=VTF8yNQm66). Oral presentation.

929 Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal,  
930 Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe  
931 Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural  
932 Information Processing Systems (NeurIPS)*, 33:9459–9474, 2020.

933 Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran  
934 You, Haotian Shen, Jiankai Sun, et al. SkillsBench: Benchmarking how well agent skills work  
935 across diverse tasks, 2026. URL <https://arxiv.org/abs/2602.12670>.

936 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding,  
937 Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui  
938 Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang.  
939 AgentBench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning  
940 Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.

941 Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and  
942 Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic  
943 evaluation of large language models. In *Proceedings of the 42nd International Conference  
944 on Machine Learning (ICML)*, volume 267, pages 48371–48392. PMLR, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.

946 Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru  
947 Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li,  
948 Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+  
949 real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*,  
950 2024. URL <https://openreview.net/forum?id=dHng200Jjr>. Spotlight.

951 Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt  
952 injection attack to tool selection in LLM agents, 2025. URL <https://arxiv.org/abs/2504.19793>. To appear at NDSS 2026.

954 Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive archi-  
955 tectures for language agents. *Transactions on Machine Learning Research (TMLR)*, 2024. URL  
956 <https://openreview.net/forum?id=1i6ZCvf1QJ>.

957 Leye Wang, Zixing Wang, and Anjie Xu. Skilltester: Benchmarking utility and security of agent  
958 skills. *arXiv preprint arXiv:2603.28815*, 2026.

959 Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang,  
960 Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su,  
961 Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham  
962 Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real-world tasks. In  
963 *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*,  
964 2025. URL <https://openreview.net/forum?id=LZnKNApvhG>.

965 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.  
966 ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Con-  
967 ference on Learning Representations (ICLR)*, 2023. URL [https://openreview.net/forum?  
968 id=WE\\_vluYUL-X](https://openreview.net/forum?id=WE_vluYUL-X).

969 Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan.  $\tau$ -bench: A benchmark for tool-  
970 agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learn-  
971 ing Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.

972 Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect  
973 prompt injections in tool-integrated large language model agents. In *Findings of the Asso-  
974 ciation for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand,  
975 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.624. URL  
976 <https://aclanthology.org/2024.findings-acl.624/>.